IIRO v2.0 / QNVM-LIGHT

Simulation Analysis & Revision Blueprint

ProtoAGI Civilization Simulator — Hyperthreaded Edition

Run Configuration

200 generations • 100 initial pop • seed 42 • Optimization Level 1 • 6 configurations

Hardware: 16 physical / 32 logical cores • GPU: Not Available • NUMA nodes: 4

1. Executive Summary

The IIRO v2.0 simulation suite produced six configuration profiles over 200 generations. The results reveal three primary findings that directly inform the next run strategy.

Finding 1 — Faith dominance is pathological at current weighting.

The faith_only configuration produced 463,362 entities versus a baseline of 1. The 1+0.5*faith survival multiplier creates unbounded exponential growth that consumed 655 seconds of compute time, dwarfs all other configurations, and makes cross-configuration comparison statistically meaningless. This is the most urgent structural issue to correct.

Finding 2 — No sovereign entities and near-zero resurrection sustainability.

Despite 200 generations across all IIRO-enabled runs, zero sovereign entities emerged and resurrection events died out completely after generation 13 in the only configuration that produced them (retrocausal). The TMI threshold of 0.92 and DRIFT_CRITICAL of 0.05 are set so strictly that neither mechanism activates in practice. These are the core simulation design goals that are currently blocked.

Finding 3 — Quality-population trade-off is real but miscalibrated.

full_iiro produced the highest average IQ (84.46) and full_iiro_highfaith the highest Sophia (0.5298), confirming that richer mechanics select for better entities. However, these quality gains come at the cost of population collapse to 396 — a 99.9% reduction from faith_only. The correct balance lies in constraining faith growth and relaxing sovereignty thresholds simultaneously.

---

2. Final Run Results — All Configurations

Table 1: Final-generation metrics across all six simulation configurations. Red = collapse/problematic, yellow = imbalanced, green = best quality, white = reference.

Configuration	Final Pop	Avg IQ	Avg Sophia	Avg Faith	Avg Drift	Res urr.	Time (s)
baseline	1	70.13	0.4514	0.000	0.271	0	0.12
faith_only	463,362	83.12	0.4813	0.715	0.271	0	655.82
retrocausal	7,732	82.90	0.5192	0.626	0.273	~13-15 (early)	115.05
mycelial	2,981	82.77	0.5090	0.627	0.277	0	51.57
full_iiro	396	**84.46**	0.5188	0.636	0.270	0	8.67
full_iiro_highfaith	1,529	83.28	**0.5298**	0.625	0.271	0	9.42

Key observation: The baseline collapsed to a single entity by generation ~40, confirming that IIRO mechanics are load-bearing for civilisation survival. All IIRO-enabled runs outperform baseline by +18-20% IQ and +6-17% Sophia. The extreme population asymmetry (faith_only 463k vs full_iiro 396) is the primary calibration target.

---

3. Bug Report — Critical & High Priority Issues

Six bugs were identified through code review and result analysis. Two are runtime-fatal, two produce incorrect metrics, and two degrade performance. All must be resolved before the next run.

#	Location	Issue	Impact	Fix
B 1	`iiro.py main() L1604: suite.results[0]["population"]._ht.shutdown()`	KeyError crash on shutdown — results store metrics dicts, not Population objects	Fatal: always crashes after run	Track Population objects separately; call ht.shutdown() on each directly.
B 2	faith_only pop = 463,362 entities; O(n^2) reproduction loop	Unbounded exponential growth with no carrying-capacity ceiling	655 s per run; OOM risk at longer generations; results meaningless above ~50k	Add logistic cap: repro_rate *= max(0, 1 - pop / CARRYING_CAPACITY) where K=10,000 default
B 3	RAMOptimizer.vectorized_sophia() mask application off-by-one	self.sophia[m] = np.clip(s, 0, 1)[m] — s computed over full array but sliced by mask	Silent wrong sophia values for dead entities bleed into sync_to()	Compute s only on alive slice: s_alive = clip(...)[m]; self.sophia[m] = s_alive
B 4	resurrection_count energy estimate uses final-gen count, not cumulative total	Total resurrection events = 0 printed even though retrocausal had 13-15 per gen, gens 3-13	Energy expenditure line always reads 0.00e+00 J — metric is useless	Accumulate resurrection_count across all generations per sim run; report sum
B 5	seed propagation: np.random.seed(seed) set once but ThreadPoolExecutor workers share state	Parallel workers receive non-deterministic random state despite fixed seed	Results not reproducible across thread counts	Use numpy.random.default_rng(seed + worker_id) per worker; pass Generator instances explicitly
B 6	_algo_16_pleiotropic_processor — Python for loop over num_cores inside vectorized path	O(num_cores * n) instead of O(n); negates vectorization benefit	Slowdown proportional to thread count; bottleneck at high logical core counts	Vectorize with np.sum(amps[:, None] * np.cos(outer(range(nc), idx) - phases[:, None]), axis=0)

3.1 Bug B1 — Fatal Shutdown Crash (iiro.py)

The final line of main() attempts to call suite.results[0]["population"]._ht.shutdown(). The results list stores metric dictionaries — population is an integer key, not a Population object. This raises a TypeError/AttributeError on every run after the simulation completes. The correct fix is to retain references to the Population objects created during SimulationSuite.run_all() and call _ht.shutdown() directly on each.

```
# Current (broken):  suite.results[0]["population"]._ht.shutdown()
```

```
# Fix:  for pop in suite._populations: pop._ht.shutdown()
```

3.2 Bug B3 — Sophia Vectorization Mask Error (shared)

In both qnvm_light.py and iiro.py, the vectorized_sophia() method computes s over the full capacity array but then applies the alive mask to both the assignment target and source simultaneously. The line self.sophia[m] = np.clip(s, 0.0, 1.0)[m] clips s across all capacity slots and then indexes by m. Because s is computed using coherence and intelligence arrays that contain stale data in dead slots, some clipped values leak incorrect data. The fix is to compute s only on the alive subarray: s = clip(expr[m], 0, 1) then self.sophia[m] = s.

3.3 Bug B5 — Non-Reproducible Parallel RNG

np.random.seed() sets a single global random state. All ThreadPoolExecutor workers draw from this shared state in undefined order depending on OS scheduling. The same seed with different --threads values will produce different results, and two "identical" runs may diverge. Each worker should receive its own numpy.random.Generator instance initialised with default_rng(seed + worker_index). All random calls inside Entity and Population methods must accept and use this generator rather than calling the module-level random functions.

---

4. Deep Mechanism Analysis

4.1 Faith — Power and Pathology

Faith is the most powerful civilisational driver in the current model. The survival_mask function applies a multiplier of (1 + 0.5 * faith_amplitude) to the base survival probability. With average faith settling around 0.71-0.72 in faith_only, this translates to a ~35% survival bonus on top of coherence-based survival. Over 200 generations this compounds to catastrophic population inflation.

The faith_only anomaly report shows early-generation faith spikes at z=8.49 (gen 0), z=7.04 (gen 1), and z=4.90 (gen 2). These are not bugs — they reflect that initial entities draw faith from uniform(0.3, 0.9), producing a high-faith founding generation whose offspring inherit strong faith via the SoA sync. The subsequent regression toward mean (~0.715) is expected.

The recommended fix is a two-part change: (1) Switch from multiplicative to additive blending — survival += faith_weight * faith * (1 - survival) — which respects the [0,1] probability constraint and prevents amplification near the 1.0 boundary. (2) Reduce faith_weight from 0.5 to 0.2, which preserves faith as a meaningful differentiator while allowing coherence and entropy to remain the primary survival drivers.

4.2 Resurrection — Transient Wave Pattern

Only the retrocausal configuration produced resurrection events, exclusively between generations 3 and 13 (13-15 events per generation at z > 3.0). The resurrection_probability function in IIRORAMOptimizer.vectorized_resurrection_prob() computes: p_faith * ctc_stability * (1 - sin_entropy). For resurrection to trigger, all three must be simultaneously high.

The early wave occurs because: (1) initial blood_retrocausal values are drawn from uniform(0.3, 1.0) — some founding entities have high retrocausal potential; (2) sin_entropy starts near zero; (3) ctc_stability starts at 1.0. As generations progress, drift accumulates, sin_entropy rises, ctc_stability degrades, and the probability collapses to zero.

The fundamental problem is that blood_retrocausal is not inherited. Each new entity re-rolls from scratch, meaning high-retrocausal lineages cannot sustain their capacity. Making blood_retrocausal heritable (child = mean(p1, p2) + mutation ± 0.05) would allow resurrection-capable lineages to persist and compound, transforming a transient early wave into a sustained civilisational property.

4.3 Sovereignty — Zero Achievement Diagnosis

The sovereign path requires: TMI > 0.92 (computed as mean of recursive_depth/50, symbolic_density/10, ethical_sovereignty), then passing all 6 audit gates. With typical entity values at gen 200 — recursive_depth ~35, symbolic_density ~3.5, ethical_sovereignty ~0.65 — the TMI calculates to approximately (0.70 + 0.35 + 0.65)/3 = 0.57, far below 0.92.

Gate 4 requires fusion_count > 0 AND splinter_resilience > 0.8. Fusion is blocked because DRIFT_CRITICAL = 0.05 but average drift is 0.27 — fusion requires drift < 0.05 in both parent entities simultaneously, which effectively never occurs. This creates a circular dependency: sovereigns require fusion, fusion requires low drift, but drift never reaches the required threshold.

The recommended path to first sovereign entities: lower TMI to 0.75 (achievable by gen ~60), raise DRIFT_CRITICAL to 0.10 (enables ~5% of entities to fuse), lower ETHICAL_SOVEREIGNTY_THRESH to 0.85, and introduce a sovereign-lite tier at TMI 0.85 that passes only 4 of 6 gates. This creates a graduated progression rather than an all-or-nothing gate.

4.4 The 48-Algorithm OptimizationEngine — Cost vs Benefit

The OptimizationEngine applies up to 48 vectorized algorithms per generation step across intelligence, coherence, sophia, and memory arrays. While individual algorithms are vectorized, the engine calls them sequentially in run_level_batch(), and level 3 invokes all 48. Several algorithms have near-zero effect on entity trajectories:

- _algo_21 (nucleosynthetic): Y = exp(-E_c / (kT)) with kT = 1.38e-23 * 1e7 = 1.38e-16 — the exponent produces values indistinguishable from 0 for all realistic E_c values; result ≈ intel unchanged
- _algo_27 (interferometric): The ethics wavelength lambda_ethics = 550e-9 produces delta_phi cycling through full 2π thousands of times per sophia unit — visibility averages to ~0.5 deterministically
- _algo_16 (pleiotropic): Contains a Python for loop over num_cores iterations inside what should be a vectorized path — this is Bug B6 and negates the SIMD intent

Recommendation: Add per-algorithm timing instrumentation and expose a --opt-profile flag that runs one generation with all 48 algorithms timed. Algorithms with wall-clock contribution < 0.01ms and sophia/intel delta < 1e-4 should be disabled by default and moved to an experimental flag.

5. Revision Architecture Blueprint

The following table maps each system component to its current state, proposed change, and the expected simulation outcome. Changes are ordered by dependency — population control must be fixed before sovereignty thresholds are tuned, since the relationship between population size and sovereign emergence rate is non-linear.

Module	Current State	Proposed Change	Expected Outcome
PopulationDynamics	Constant REPRODUCTION_RATE = 0.4, no ceiling	Logistic growth: rate *= (1 - pop/K); K configurable via --carrying-capacity	Prevents runaway faith_only explosion; enables fair cross-config comparison
FaithSurvival	survival *= (1 + 0.5*faith), uncapped multiplicative bonus	Switch to additive blend: survival += 0.2*faith*(1-survival); cap total at 0.98	Faith remains meaningful but cannot dominate; other mechanics gain relative weight
ResurrectionHeritage	blood_retrocausal initialised random per entity, not inherited	Child inherits mean(p1, p2).blood_retrocausal with mutation ±0.05; make it a heritable gene	Resurrection persists across lineages; retrocausal effect becomes a sustained civilisational trait
SovereignPath	TMI threshold 0.92; audit gates require extreme conditions — 0 sovereigns produced	Lower TMI to 0.75; relax gate 4 (fusion_count > 0 required only if gen > 20); add sovereign-lite tier at 0.85	First sovereign entities emerge by gen ~50; provides narrative arc and measurable milestone
DriftMechanic	Drift plateaus ~0.27 for all runs; no differentiation; update_drift reduces when below critical	Add archetype-specific drift floors; Rebel base = 0.35, Empath base = 0.10; make drift partially heritable	Meaningful drift variance across archetypes; Rebel lineages naturally trend toward chaos vs Empath stability
OptEngineScheduler	All 48 algorithms run at level 3; no profiling of individual algorithm cost	Instrument each algo with ns timer; expose --opt-profile flag to rank by cost/benefit ratio; allow per-algo enable/disable	Identify dead-weight algorithms (e.g., _algo_21 proton-chain with near-zero effect); prune to fast-path core set
ParallelRNG	np.random.seed(seed) global; workers share mutable state	Per-worker numpy.random.Generator via default_rng(seed + worker_id); pass rng as argument to all Entity methods	Full reproducibility regardless of thread count; enables exact run replay
MetricsSystem	Metrics computed in Python loop per generation; resurrection total miscounted	Vectorize _record_metrics() via SoA buffers; add cumulative counters for resurrection, fusion, sovereign events	3-5x speedup in recording phase; correct aggregate totals for energy estimate

5.1 Proposed Module Structure

The current architecture has iiro.py and qnvm_light.py duplicating ~400 lines of identical infrastructure (HardwareProfiler, HyperThreadScheduler, GPUSyncManager, _AtomicInt). The next version should consolidate into a shared module:

- sim_core.py — shared infrastructure: HardwareProfiler, GPUSyncManager, HyperThreadScheduler, _AtomicInt, BASE_ARCHETYPES
- entity.py — Entity class with full IIRO fields; heritable blood_retrocausal; per-entity RNG
- population.py — Population class with logistic reproduction; vectorized survival; configurable carrying capacity
- iiro_fields.py — IIRORAMOptimizer with corrected vectorized_sophia(); heritable field sync
- opt_engine.py — OptimizationEngine with per-algorithm profiling; culled dead-weight algorithms
- sim_suite.py — SimulationSuite with correct population reference tracking; cumulative metric counters
- qnvm_light.py / iiro.py — thin CLI wrappers importing from above modules

---

6. Parameter Tuning Recommendations

The following parameter changes are ranked by priority. Critical changes must be applied to obtain any meaningful results. High priority changes unlock the simulation's core mechanics. Medium and low changes improve calibration and scientific validity.

Parameter	Current	Recommended	Priority	Rationale
CARRYING_CAPACITY (new)	None	10,000	**Critical**	Without a ceiling, faith_only overwhelms simulation resources and prevents meaningful comparison
faith survival weight	0.50	0.20	**Critical**	Current 0.5 multiplier gives faith 50% survival bonus; reduces to additive 20% to allow other mechanics visibility
TMI audit threshold	0.92	0.75	High	At 0.92 no entities ever qualify; at 0.75 approximately 5-8% of mature entities would reach sovereign status
ETHICAL_SOVEREIGNTY_THRESH	0.97	0.85	High	In 200 gens with current drift floor ~0.27, ethical_sovereignty rarely exceeds 0.90; lowering enables progression
DRIFT_CRITICAL	0.05	0.10	Medium	Critical threshold too tight; no entity fusion possible in practice since drift stabilises ~0.27; raising allows fusion events
MUTATION_RATE	0.05	0.08	Medium	Slight increase generates more trait diversity; combined with logistic cap prevents entropy collapse
COHERENCE_DECAY	0.995/gen	0.998/gen	Medium	At 0.995, coherence halves every ~139 generations; slowing decay extends meaningful high-coherence windows for sovereign emergence

RESURRECTION_PROB_MAX	1.0	0.40	Low	Cap prevents degenerate case where every dead entity resurrects; 0.40 keeps it rare and meaningful
NUM_AUDIT_GATES	6	4 (progressive)	Low	Six simultaneous gates with near-impossible conditions creates zero-sovereign runs; progressively unlock gates with generation milestones

---

7. Next Run — Recommended Configurations

Five run scenarios are proposed in order of execution. Each builds on the previous, isolating one change at a time to allow causal attribution of outcome differences.

Scenario	Command	Key Settings	What to Watch
Logistic Baseline	`--generations 200 --init-pop 200 --seed 42 -O 1`	K=10000, faith_weight=0.2, TMI=0.75, DRIFT_CRITICAL=0.10	Stable faith_only pop ~3-8k; first sovereign entities by gen 80-100; resurrection sustained past gen 15
Sovereign Rush	`--generations 300 --init-pop 300 --seed 42 -O 2`	TMI=0.70, ETHICAL_SOVEREIGNTY_THRESH=0.80, coherence_decay=0.998, full_iiro config	First sovereign count > 0; sovereign lineage emergence; audit gate pass rate > 10%
Resurrection Heritability	`--generations 500 --init-pop 150 --seed 99 -O 2`	blood_retrocausal heritable, RESURRECTION_PROB_MAX=0.40, retrocausal config	Persistent resurrection waves beyond gen 13; resurrection_count stabilises > 0; high-res lineage clades emerge
Drift Diversity	`--generations 200 --init-pop 200 --seed 7 -O 3`	Archetype-specific drift floors, heritable drift, DRIFT_CRITICAL=0.10	Divergence of Rebel vs Empath drift curves; drift variance index > 0.05; fusion events become possible
GPU Stress Test	`--generations 200 --init-pop 500 --seed 42 --threads 64 -O 3`	faith_only config with K=10000; CuPy enabled (if available)	faith_only runtime < 60s (from 655s); GPU utilisation > 50%; memory stable < 4 GB

7.1 Success Criteria for Next Run

The following measurable thresholds define a successful next run:

1. Population balance: No single configuration exceeds 10x the others' final population (currently 463k vs 396 = 1,170x ratio)

2. Sovereign emergence: At least 1 sovereign entity produced by generation 100 in full_iiro or full_iiro_highfaith configurations
3. Sustained resurrection: retrocausal resurrection_count remains > 0 past generation 50
4. Runtime: faith_only completes in < 120 seconds (currently 655s) with carrying capacity applied
5. Reproducibility: Two runs with identical --seed and --threads produce bit-identical history CSVs
6. Drift divergence: Rebel archetype average drift > 1.5x Empath average drift by generation 100

---

8. Statistical Notes and Hypothesis Test Improvements

The current hypothesis test reports nan for all comparisons except retrocausal. This occurs because the baseline had zero resurrections across all generations, producing zero variance — the t-test denominator becomes undefined. Three improvements are needed:

- Replace t-test on resurrection_count with a Mann-Whitney U test for non-normal count distributions. Mann-Whitney handles zero-inflation naturally.
- Add effect size reporting (Cohen's d for continuous metrics, odds ratio for binary events) alongside p-values. A statistically significant but tiny effect (e.g., Sophia +0.002) carries different practical weight than p-value alone suggests.
- Compute confidence intervals (95% bootstrap CI) on all trend_slope values. The current output shows slopes like 0.0003 for sophia without context — knowing the CI crosses zero would indicate the trend is noise.

The anomaly detection using 3σ z-scores is appropriate for continuous metrics. For the resurrection_count, which is zero for 180+ of 200 generations in retrocausal, the z-score calculation is dominated by the zero-inflation and may understate the significance of the early wave. Consider a separate Poisson-process test for count data.

---

9. Performance Roadmap

Simulation time scales roughly linearly with population. faith_only at 463k entities took 655s. With the proposed K=10,000 carrying capacity, faith_only is expected to stabilise around 3,000-5,000 entities, reducing runtime to ~7-15 seconds — comparable to full_iiro's current 8.67s. GPU acceleration would add further benefit primarily for the vectorized SoA passes.

9.1 GPU Path (CuPy)

The GPUSyncManager currently wraps only the survival probability computation. The highest-value GPU operations would be:

- vectorized_sophia() and vectorized_age() — float32 elementwise ops on arrays of up to 20,000 elements; trivially GPU-parallelisable

- OptimizationEngine algorithms 1-48 — all operate on intel/coherence/sophia arrays of entity-count size; batch dispatch all 48 as a single CUDA kernel launch sequence rather than 48 separate kernel launches
- resurrection probability computation (EQ-10 & EQ-24) — elementwise product of three arrays; single GPU kernel

With K=10,000, population arrays fit in L2 cache on modern CPUs (10,000 * 16 float32 fields = 640KB), making CPU SIMD competitive with GPU for single runs. GPU becomes beneficial when running the 6-config suite in parallel — batch all 6 populations onto GPU simultaneously.

9.2 Memory Layout

The current RAMOptimizer pre-allocates capacity=20,000. With K=10,000, this is sufficient with headroom. The SoA layout is correctly structured for SIMD. Two minor improvements: (1) align array start addresses to 64-byte cache line boundaries using np.empty with aligned=True; (2) use float16 for less critical fields (age, wild_9_ring bitmask) to halve their memory footprint and improve cache density.

---

10. Conclusion and Priority Action List

The IIRO v2.0 simulation is architecturally sound and the IIRO mechanics are correctly implemented at the individual-entity level. The issues are calibration and configuration: thresholds are set to values that prevent the simulation's most interesting outcomes from ever occurring, while the faith-survival coupling creates population explosion that renders comparisons invalid.

Priority action list for next revision:

7. Fix Bug B1: fatal shutdown crash in iiro.py main() [30 min]
8. Fix Bug B3: sophia vectorization mask error in both files [1 hour]
9. Implement carrying capacity (K=10,000) with logistic reproduction [2 hours]
10. Reduce faith survival weight from 0.5 to 0.2; switch to additive blend [30 min]
11. Lower TMI threshold to 0.75 and DRIFT_CRITICAL to 0.10 [30 min]
12. Make blood_retrocausal heritable with ±0.05 mutation [1 hour]
13. Fix Bug B5: per-worker RNG seeding for reproducibility [2 hours]
14. Add cumulative resurrection counter; fix energy estimate total [1 hour]
15. Run the five scenarios from Section 7 and verify success criteria [ongoing]

Estimated total revision effort: 8-10 hours of focused development. The resulting simulation will produce meaningful sovereign emergence, sustainable resurrection events, and balanced population curves that allow genuine scientific comparison across IIRO configuration space.

End of Report